Machine Learning Practical

Practical No. 6

Title: Logistic regression

Problem: Using logistic regression (e.g. to predict whether a credit card transaction is fraudulent or not)

Solution:

Step 1:Importing libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LogisticRegression
```

Step 2: Data Loading

```
df = pd.read_csv('C:/Users/shalinik1/OneDrive - KPMG/Desktop/colleges/datasets/creditcard.csv')

df.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267

5 rows × 31 columns

Step 3: EDA

```
df.shape
```

```
(284807, 31)
```

```
labels = df.Class.unique()

print(labels)

sizes = df.Class.value_counts().values

print(sizes)
```

```
[0 1]
[284315    492]
```

```
df.drop_duplicates(keep='first',inplace=True)
df.shape
```

```
(283726, 31)
```

```
df.isnull().sum()
```

```
Out[40]: Time      0
         V1        0
         V2        0
         V3        0
         V4        0
         V5        0
         V6        0
         V7        0
         V8        0
         V9        0
         V10       0
         V11       0
         V12       0
         V13       0
         V14       0
         V15       0
         V16       0
         V17       0
         V18       0
         V19       0
         V20       0
         V21       0
         V22       0
         V23       0
         V24       0
         V25       0
         V26       0
         V27       0
         V28       0
         Amount    0
         Class     0
         dtype: int64
```

df.describe()

	Time	V1	V2	V3	V4	V5	V6	V7	V8
count	283726.000000	283726.000000	283726.000000	283726.000000	283726.000000	283726.000000	283726.000000	283726.000000	283726.000000
mean	94811.077600	0.005917	-0.004135	0.001613	-0.002966	0.001828	-0.001139	0.001801	-0.000854
std	47481.047891	1.948026	1.646703	1.508682	1.414184	1.377008	1.331931	1.227664	1.179054
min	0.000000	-56.407510	-72.715728	-48.325589	-5.683171	-113.743307	-26.160506	-43.557242	-73.216718
25%	54204.750000	-0.915951	-0.600321	-0.889682	-0.850134	-0.689830	-0.769031	-0.552509	-0.208828
50%	84692.500000	0.020384	0.063949	0.179963	-0.022248	-0.053468	-0.275168	0.040859	0.021898
75%	139298.000000	1.316068	0.800283	1.026960	0.739647	0.612218	0.396792	0.570474	0.325704
max	172792.000000	2.454930	22.057729	9.382558	16.875344	34.801666	73.301626	120.589494	20.007208

8 rows × 31 columns

```
X=df.drop(['Class'],axis=1)
print(X.head())
Y=df['Class']
print(Y.head())
```

```
    Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V20       V21       V22       V23       V24  \
0  0.098698  0.363787  ...  0.251412 -0.018307  0.277838 -0.110474  0.066928
1  0.085102 -0.255425  ... -0.069083 -0.225775 -0.638672  0.101288 -0.339846
2  0.247676 -1.514654  ...  0.524980  0.247998  0.771679  0.909412 -0.689281
3  0.377436 -1.387024  ... -0.208038 -0.108300  0.005274 -0.190321 -1.175575
4 -0.270533  0.817739  ...  0.408542 -0.009431  0.798278 -0.137458  0.141267

        V25       V26       V27       V28  Amount
0  0.128539 -0.189115  0.133558 -0.021053  149.62
1  0.167170  0.125895 -0.008983  0.014724    2.69
2 -0.327642 -0.139097 -0.055353 -0.059752  378.66
3  0.647376 -0.221929  0.062723  0.061458  123.50
4 -0.206010  0.502292  0.219422  0.215153   69.99

[5 rows x 30 columns]
0    0
1    0
2    0
3    0
4    0
Name: Class, dtype: int64
```

Step 4: Data Splitting

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=
42)
```

Step 5: Model Building

```
logreg_model = LogisticRegression(max_iter=1000)
```

Step 6:Train the model

```
logreg_model.fit(X_train, y_train)
```

Step 7: Make predictions

```
y_pred_logreg = logreg_model.predict(X_test)
```

Step 8: Evaluate the accuracy

```
accuracy_logreg = logreg_model.score(X_test, y_test)
print("LogisticRegression Accuracy: {0:.5f}".format(accuracy_logreg))
```

```
LogisticRegression Accuracy: 0.99910
```

Practical No. 7

Title: Model Evaluation Techniques

Problem: Evaluating a classification model using metrics such as accuracy, precision, recall, and F1 score

Solution:

Step 1: Import Necessary Libraries

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

Step 2: Calculate Metrics

```
# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print(f'Accuracy: {accuracy * 100:.2f}%')


# Precision

precision = precision_score(y_test, y_pred, pos_label='positive')

print(f'Precision: {precision:.2f}')


# Recall
```

```
recall = recall_score(y_test, y_pred, pos_label='positive')
print(f'Recall: {recall:.2f}')

# F1 Score
f1 = f1_score(y_test, y_pred, pos_label='positive')
print(f'F1 Score: {f1:.2f}')

conf_matrix = confusion_matrix(y_test, y_pred, labels=['positive', 'negative'])
print('Confusion Matrix:')
print(conf_matrix)
```

Expected Output:

```
Accuracy: 85.00%
Precision: 0.87
Recall: 0.82
F1 Score: 0.84
Confusion Matrix:
[[85 15]
 [10 90]]
```

Understanding the Metrics

- **Accuracy**: The ratio of correctly predicted instances to the total instances. It gives an overall measure of how often the classifier is correct.
- **Precision**: The ratio of correctly predicted positive observations to the total predicted positives. It answers the question: "What proportion of positive identifications was actually correct?"
- **Recall (Sensitivity)**: The ratio of correctly predicted positive observations to all observations in the actual class. It answers the question: "What proportion of actual positives was identified correctly?"

- **F1 Score**: The weighted average of Precision and Recall. It is especially useful when you have an uneven class distribution.

Practical No. 8

Applying hierarchical clustering (e.g. to group customer segments based on their purchasing behavior).

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

import matplotlib.pyplot as plt

# Load dataset

data = pd.read_excel('C:/Users/shalinik1/downloads/clustering.xlsx')

# Display the first few rows

data.head()
```

	CustomerID	AnnualIncome	SpendingScore	Age	Gender	MembershipYears	PurchaseFrequency
0	1	27291	2	28	Female	12	44
1	2	23681	28	57	Male	15	25
2	3	26840	31	31	Male	4	3
3	4	20260	25	41	Male	18	43
4	5	20036	12	30	Female	11	6

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         1000 non-null   int64
 1   AnnualIncome       1000 non-null   int64
 2   SpendingScore      1000 non-null   int64
 3   Age                1000 non-null   int64
 4   Gender             1000 non-null   object
 5   MembershipYears    1000 non-null   int64
 6   PurchaseFrequency  1000 non-null   int64
dtypes: int64(6), object(1)
memory usage: 54.8+ KB
```

```
data.describe()
```

	CustomerID	AnnualIncome	SpendingScore	Age	MembershipYears	PurchaseFrequency
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	54063.598000	50.229000	43.400000	9.621000	25.021000
std	288.819436	32067.838296	27.295083	15.295292	5.513053	14.163727
min	1.000000	15035.000000	1.000000	18.000000	1.000000	1.000000
25%	250.750000	22984.000000	28.000000	30.000000	5.000000	13.000000
50%	500.500000	50004.000000	49.500000	43.000000	9.000000	25.500000
75%	750.250000	85669.750000	72.000000	57.000000	14.000000	37.000000
max	1000.000000	99994.000000	99.000000	69.000000	19.000000	49.000000

```
data['Gender']=data['Gender'].map({"Male":0,"Female":1})
```

```
data
```

	CustomerID	AnnualIncome	SpendingScore	Age	Gender	MembershipYears	PurchaseFrequency
0	1	27291	2	28	1	12	44
1	2	23681	28	57	0	15	25
2	3	26840	31	31	0	4	3
3	4	20260	25	41	0	18	43
4	5	20036	12	30	1	11	6
...	...	...	...	...	...	...	...
995	996	85750	94	41	0	1	31
996	997	89694	61	46	1	6	49
997	998	96052	58	34	1	10	45
998	999	73329	68	42	0	13	17
999	1000	88332	95	19	1	19	43

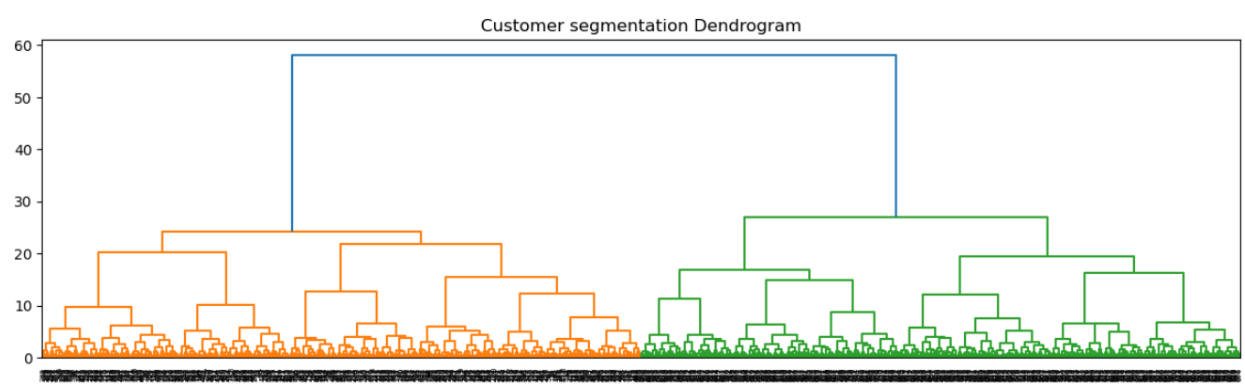
```
df=data.drop("CustomerID",axis=1)
```

```
df
```

	AnnualIncome	SpendingScore	Age	Gender	MembershipYears	PurchaseFrequency
0	27291	2	28	1	12	44
1	23681	28	57	0	15	25
2	26840	31	31	0	4	3
3	20260	25	41	0	18	43
4	20036	12	30	1	11	6
...	...	...	...	...	...	...
995	85750	94	41	0	1	31
996	89694	61	46	1	6	49
997	96052	58	34	1	10	45
998	73329	68	42	0	13	17
999	88332	95	19	1	19	43

1000 rows × 6 columns

```
scaler = StandardScaler()

data_scaled = scaler.fit_transform(data[['AnnualIncome', 'SpendingScore', 'Age',
'MembershipYears', 'PurchaseFrequency']])

# Perform hierarchical clustering

Z = linkage(data_scaled, method='ward')

plt.figure(figsize=(10, 27))

plt.title('Customer segmentation Dendrogram')

dendrogram(Z)

plt.show()
```



```
# Form clusters

max_d = 50  # This is an example threshold; adjust based on your dendrogram

clusters = fcluster(Z, max_d, criterion='distance')


# Add cluster labels to the original data

df['Cluster'] = clusters
```

	AnnualIncome	SpendingScore	Age	Gender	MembershipYears	PurchaseFrequency	Cluster
0	27291	2	28	1	12	44	1
1	23681	28	57	0	15	25	1
2	26840	31	31	0	4	3	1
3	20260	25	41	0	18	43	1
4	20036	12	30	1	11	6	1

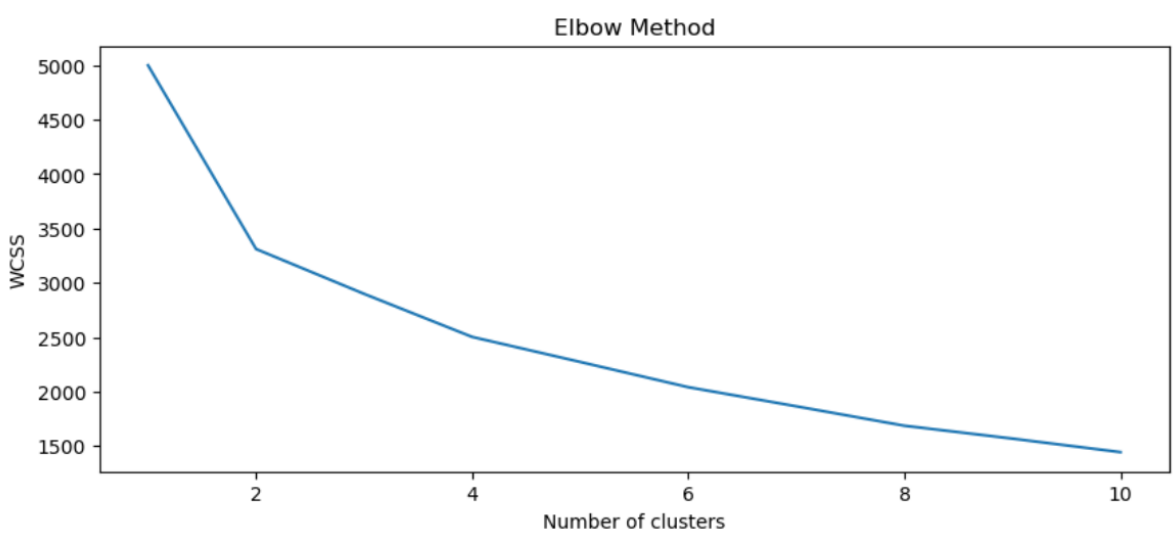
```
# Group by cluster and calculate mean values for each feature

cluster_summary = df.groupby('Cluster').mean()

cluster_summary
```

Cluster	AnnualIncome	SpendingScore	Age	Gender	MembershipYears	PurchaseFrequency
1	22768.886	26.834	43.692	0.496	9.634	24.748
2	85358.310	73.624	43.108	0.518	9.608	25.294

Practical 9. Implementing the K-means clustering algorithm (e.g. to identify distinct clusters in a customer demographic dataset).

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt
```

```
# Load dataset

data = pd.read_excel('C:/Users/shalinik1/downloads/clustering.xlsx')

scaler = StandardScaler()

data_scaled = scaler.fit_transform(df[['AnnualIncome', 'SpendingScore', 'Age',
'MembershipYears', 'PurchaseFrequency']])

wcss = []  # Within-Cluster Sum of Squares (WCSS)

for i in range(1, 11):

    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=42)

    kmeans.fit(data_scaled)

    wcss.append(kmeans.inertia_)


# Plot the Elbow graph

plt.figure(figsize=(10, 7))

plt.plot(range(1, 11), wcss)

plt.title('Elbow Method')

plt.xlabel('Number of clusters')

plt.ylabel('WCSS')

plt.show()
```




```
optimal_clusters = 3
```

```
# Apply K-means clustering with the optimal number of clusters

kmeans = KMeans(n_clusters=optimal_clusters, init='k-means++', max_iter=300, n_init=10,
random_state=42)

clusters = kmeans.fit_predict(data_scaled)

# Add cluster labels to the original data

df['Cluster'] = clusters

# Group by cluster and calculate mean values for each feature

cluster_summary = df.groupby('Cluster').mean()

cluster_summary
```

Cluster	AnnualIncome	SpendingScore	Age	Gender	MembershipYears	PurchaseFrequency
0	22768.886000	26.834000	43.692000	0.496000	9.634000	24.748000
1	84972.957627	73.889831	42.237288	0.563559	11.127119	12.296610
2	85702.791667	73.386364	43.886364	0.477273	8.250000	36.912879

Practical 10. Utilizing Principal Component Analysis (PCA) for dimensionality reduction to improve the efficiency and interpretability of a model.

```
import pandas as pd

# File path

file_path = r"D:\KPMG\LRT\pca_business.csv"


# Load dataset

df = pd.read_csv(file_path)

# Display dataset info

print(df.shape)

print(df.head())

print(df.describe())
```

```
(1500, 21)
   Age  Income  Household_Size  Grocery_Spend  Electronics_Spend  \
0   56   81268               3     384.271239         112.947926
1   69  112201               5     197.170090        1094.473652
2   46   58243               2     450.525661        1850.875445
3   32   77384               4      87.935592        1642.947461
4   60   51653               4     413.235193        1667.901605

   Clothing_Spend  Dining_Spend  Entertainment_Spend  Subscription_Spend  \
0      699.160044    372.728250           420.537638           10.386942
1      224.385909    166.196176           619.541437           58.561322
2      829.873253    104.499047           422.928081           25.010014
3      457.440170    397.532426           134.892946           95.140355
4      229.536377    170.020953           108.541529           36.876390

   Website_Visits  ...  Years_as_Customer  Total_Transactions  \
0              26  ...                  7                  47
1              31  ...                  6                  53
2              32  ...                 14                 360
3              21  ...                  7                  83
4              18  ...                  7                 447

   High_Value_Purchases  Discount_Usage  Return_Rate  Customer_Service_Calls  \
0                     4        0.170332     0.173493                       6
1                    17        0.561855     0.250054                       7
2                    16        0.077320     0.058694                       3
3                     3        0.949521     0.048543                       7
4                     6        0.603314     0.225195                       0

   Loyalty_Score  Referral_Count  Complaints  Customer_Segment
0      37.378504               0           4                 1
1      51.211997              10           1                 2
2      24.034248              17           2                 3
3      40.010480              14           4                 2
4      54.216381               8           2                 4
```

```
               Age         Income  Household_Size  Grocery_Spend  \
count  1500.000000    1500.000000     1500.000000    1500.000000
mean     43.732000   70364.540667        3.002667     272.926269
std      15.071771   28443.248339        1.420330     129.139211
min      18.000000   20060.000000        1.000000      50.083938
25%      31.000000   46650.250000        2.000000     163.742153
50%      44.000000   69687.000000        3.000000     272.220949
75%      56.000000   94871.750000        4.000000     386.546052
max      69.000000  119986.000000        5.000000     498.930270

       Electronics_Spend  Clothing_Spend  Dining_Spend  Entertainment_Spend  \
count        1500.000000     1500.000000   1500.000000          1500.000000
mean         1039.974411      529.417499    262.676531           403.418103
std           552.143456      271.864174    139.944142           224.731915
min           101.192282       51.204673     20.650652            10.146304
25%           560.249408      301.016625    145.427638           215.686891
50%          1038.016557      533.349168    265.836437           402.177100
75%          1518.875906      758.803868    385.197769           590.481575
max          1998.194046      999.856782    499.875811           799.602117

       Subscription_Spend  Website_Visits  ...  Years_as_Customer  \
count         1500.000000     1500.000000  ...        1500.000000
mean            52.215072       24.482667  ...           7.461333
std             27.548599       14.324565  ...           4.064257
min              5.103267        1.000000  ...           1.000000
25%             28.993147       12.000000  ...           4.000000
50%             52.038388       24.000000  ...           7.000000
75%             75.626014       37.000000  ...          11.000000
max             99.965384       49.000000  ...          14.000000
```

```
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

# Define features and target

X = df.drop(columns=['Customer_Segment'])

y = df['Customer_Segment']

# Split data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Scale the features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report


# Train a Random Forest model
```

```
clf = RandomForestClassifier(n_estimators=100, random_state=42)

clf.fit(X_train_scaled, y_train)

# Predict on test set

y_pred = clf.predict(X_test_scaled)

# Evaluate

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

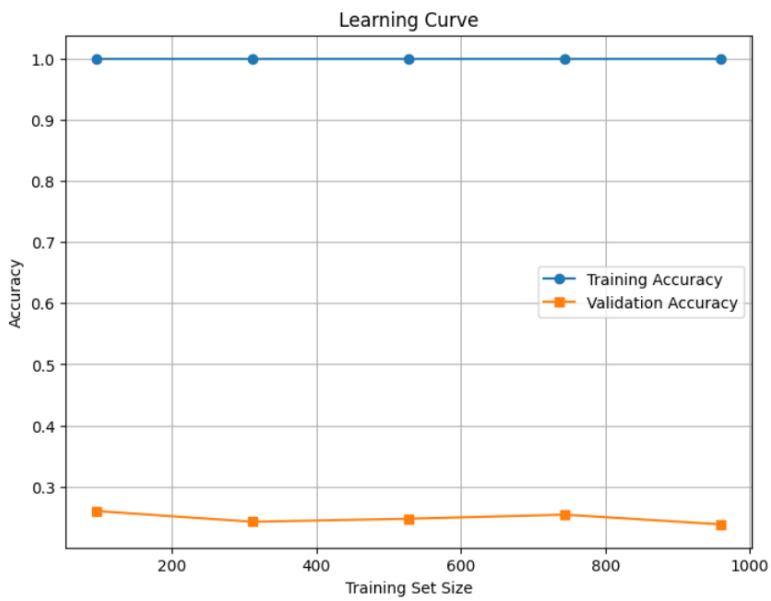
```
Accuracy: 0.26666666666666666

Classification Report:
               precision    recall  f1-score   support

           1       0.24      0.21      0.23        71
           2       0.25      0.23      0.24        75
           3       0.31      0.38      0.34        79
           4       0.25      0.24      0.24        75

    accuracy                           0.27       300
   macro avg       0.26      0.26      0.26       300
weighted avg       0.26      0.27      0.26       300
```

```
# Train accuracy

y_train_pred = clf.predict(X_train_scaled)

train_acc = accuracy_score(y_train, y_train_pred)


# Test accuracy

test_acc = accuracy_score(y_test, y_pred)


print(f"Training Accuracy: {train_acc:.4f}")

print(f"Testing Accuracy: {test_acc:.4f}")
```

```
Training Accuracy: 1.0000
Testing Accuracy: 0.2667
```

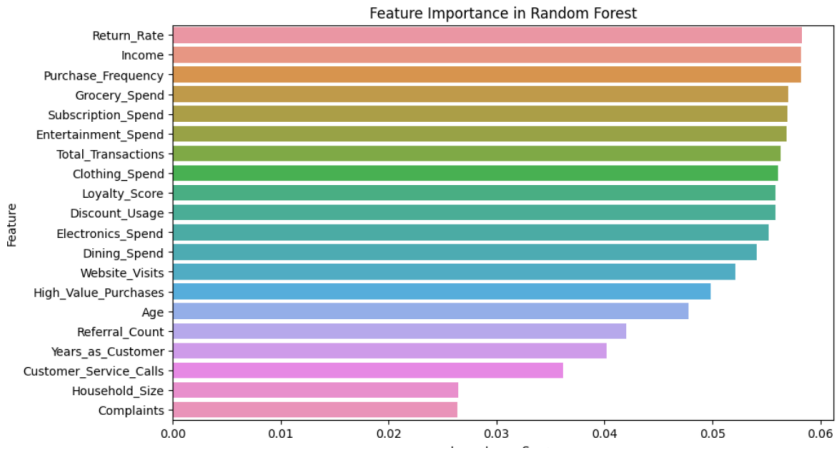
```
import matplotlib.pyplot as plt

import numpy as np

from sklearn.model_selection import learning_curve


# Compute learning curve

train_sizes, train_scores, test_scores = learning_curve(clf, X_train_scaled, y_train, cv=5, scoring="accuracy")


# Calculate mean and std deviation

train_mean = np.mean(train_scores, axis=1)
```

```
test_mean = np.mean(test_scores, axis=1)

# Plot learning curve
plt.figure(figsize=(8, 6))
plt.plot(train_sizes, train_mean, label="Training Accuracy", marker='o')
plt.plot(train_sizes, test_mean, label="Validation Accuracy", marker='s')

plt.xlabel("Training Set Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve")
plt.legend()
plt.grid()
plt.show()
```



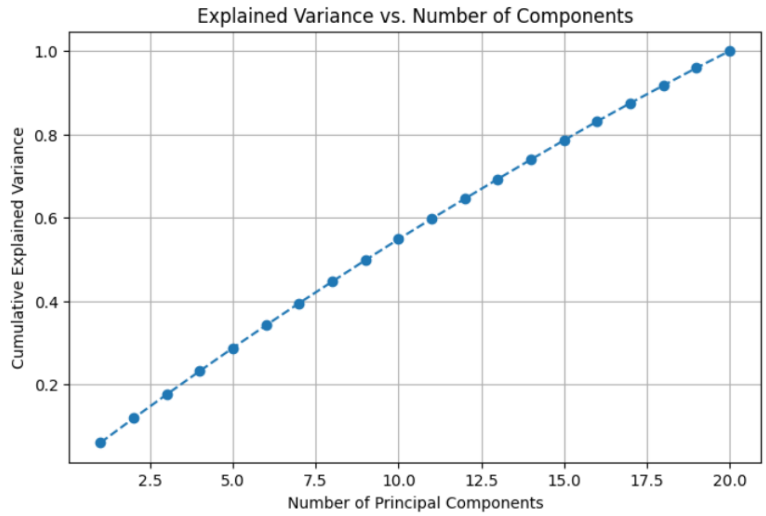

```
import seaborn as sns

# Get feature importances
feature_importances = pd.Series(clf.feature_importances_, index=X.columns).sort_values(ascending=False)

# Plot feature importance
plt.figure(figsize=(10, 6))
sns.barplot(x=feature_importances.values, y=feature_importances.index)
plt.xlabel("Importance Score")
plt.ylabel("Feature")
```

```
plt.title("Feature Importance in Random Forest")

plt.show()
```

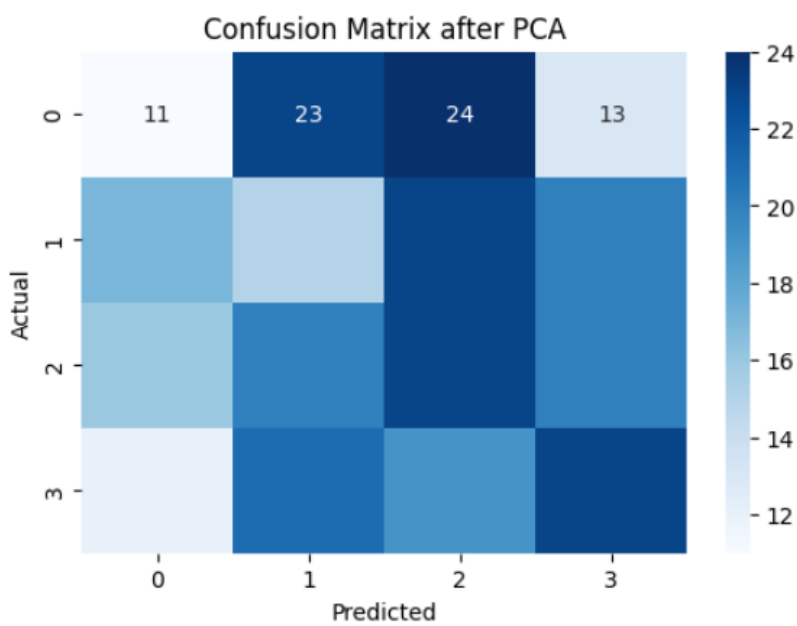



```
from sklearn.preprocessing import StandardScaler


# Standardize features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)  # X is your dataset without 'Customer_Segment'


from sklearn.decomposition import PCA

import numpy as np

import matplotlib.pyplot as plt


# Apply PCA (initially without specifying number of components)

pca = PCA()

X_pca = pca.fit_transform(X_scaled)


# Explained variance ratio

explained_variance = np.cumsum(pca.explained_variance_ratio_)


# Plot cumulative explained variance

plt.figure(figsize=(8, 5))

plt.plot(range(1, len(explained_variance) + 1), explained_variance, marker='o', linestyle='--')

plt.xlabel("Number of Principal Components")

plt.ylabel("Cumulative Explained Variance")

plt.title("Explained Variance vs. Number of Components")

plt.grid()
```

```
plt.show()
```




```
# Keep 95% variance
pca = PCA(n_components=0.95)
X_pca_reduced = pca.fit_transform(X_scaled)

print(f"Original Features: {X.shape[1]}, Reduced Features: {X_pca_reduced.shape[1]}")
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Split the PCA Transformed Data
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_pca_reduced, y, test_size=0.2, random_state=42, stratify=y)

# Step 2: Train the Model (Random Forest Classifier)
clf_pca = RandomForestClassifier(n_estimators=100, random_state=42)
clf_pca.fit(X_train_pca, y_train)

# Step 3: Make Predictions
y_pred_pca = clf_pca.predict(X_test_pca)

# Step 4: Evaluate Performance
accuracy_pca = accuracy_score(y_test, y_pred_pca)
print(f"Accuracy after PCA: {accuracy_pca:.4f}")
```

```
# Detailed Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_pca))

# Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred_pca), annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix after PCA")
plt.show()
```

```
Accuracy after PCA: 0.2400

Classification Report:
              precision    recall  f1-score   support

           1       0.20      0.15      0.17        71
           2       0.19      0.20      0.19        75
           3       0.26      0.29      0.27        79
           4       0.30      0.31      0.30        75

    accuracy                           0.24       300
   macro avg       0.24      0.24      0.24       300
weighted avg       0.24      0.24      0.24       300
```